



Nix for Polyglot, Reproducible Data Science Workflows

Bruno Rodrigues 

Ministry of Research and Higher Education

Philipp Baumann 

Alliance SwissPass

Abstract

Reproducible analysis requires more than clean, well-documented code; it demands the precise management of software dependencies, computational environments, and workflow execution. While researchers can combine tools like **Docker**, **renv** for R packages (or **uv** for Python packages) and **Make** to address these needs, the resulting toolchain is often complex and fragile, requiring the coordination of multiple disparate systems. This paper introduces a unified framework built on the **Nix** package manager, made accessible through two R packages. The first, **rix**, generates declarative **Nix** expressions to define version-pinned, reproducible environments that encompass R, Python, Julia, and system-level dependencies. The second, **rixxpress** (and its Python port, **ryxxpress**), leverages these environments to orchestrate polyglot pipelines where each computational step runs in a hermetically sealed, language-specific environment with automatic caching. By treating the entire computational environment as code, this integrated approach greatly increases the degree of reproducibility, supports collaboration across heterogeneous systems, and helps ensure that analyses remain executable long into the future.

Keywords: reproducibility, R, Python, Julia, Nix.

1. Introduction

1.1. The reproducibility continuum

Peng (2011) introduced reproducibility as a *continuum*, ranging from a paper-only description to fully linked and executable code and data. We extend this continuum to the *computational environment*: the complete set of software required to execute an analysis. At the minimal end, authors name the language used; more carefully, they specify versions and packages. Rarely, however, do authors document the operating system or provide step-by-

step installation instructions—even though platform differences can lead to divergent results (Bhandari Neupane, Neupane, Luo, Yoshida, Sun, and Williams 2019).

This paper focuses on two critical but often overlooked aspects of reproducibility: *computational environment management* and *workflow orchestration*. We present a comprehensive framework addressing both challenges using the **Nix** package manager (Dolstra, De Jonge, and Visser 2004), making it accessible to researchers through two R packages: **rix** (Rodrigues and Baumann 2025) and **rixpress** (Rodrigues 2025). Before introducing these packages, we survey existing tools and their limitations to contextualise our contributions.

1.2. The polyglot challenge

Indeed, there is evidence that the era of a single tool dominating an entire analytical pipeline is giving way to a more pragmatic approach where researchers combine the complementary strengths of multiple languages (Peng, Wang, Ma, Leong, Wakefield, Melott, Chiu, Du, and Weinstein 2018). This trend is particularly evident in the synergistic use of R, with its rich ecosystem for statistical modeling, Python, with its extensive libraries for machine learning, and Julia for complex simulations through the use of interoperability libraries such as **Python-Call** (calling Python from Julia), **JuliaCall** (calling Julia from Python or R), **reticulate** (calling Python from R), **rpy2** and **ryp** (calling R from Python) (Osborne, Pivarski, and Ling 2024; Navarro, Koneva, Sánchez-Macián, and Hernández 2024). Recent survey data confirms this trend, showing that research scientists use, on average, nearly two programming languages in their work, with the R and Python combination being the most prevalent (Chen, Wong, Sharif, and Peruma 2025). This creates a significant challenge for reproducibility: a project using R, Python, and system tools like **Quarto** or $\{\text{L}^{\text{A}}\text{T}^{\text{E}}\text{X}\}$ requires coordinating multiple, disparate package managers, each with its own configuration and potential for conflict and using interoperability libraries.

The demand for solutions that can manage this complexity is not merely academic; it is reflected in major strategic shifts within the industry. A prominent example is the evolution of Posit (formerly RStudio), which has pivoted from its origins as a premier R-centric service and software supplier to a company that treats Python as a first-class citizen alongside R. This industry trend, coupled with the proliferation of academic research into polyglot tooling, underscores a critical reality: the central challenge for modern reproducible science is no longer *if* polyglot workflows should be used, but *how* they can be managed in a unified, efficient, and reproducible manner.

1.3. Our contribution

The **Nix** package manager provides the capabilities researchers need: complete environment management, multi-language support, interactive workflows, step-level isolation, deterministic builds, and accessibility without systems administration expertise. However, **Nix** has a steep learning curve due to its own functional programming language, which creates a barrier to adoption.

Our contribution is not a new reproducibility engine, but a user-friendly interface that makes **Nix** accessible to data scientists. We developed two R packages and a supporting infrastructure

layer:

- **rix** makes **Nix** accessible to the average data scientist. It provides an R-native (and Python-native via **ryxpress**) declarative interface to **Nix**, generating **Nix** expressions from intuitive function calls and eliminating the need to learn the **Nix** language. It handles environment definitions including R, Python, Julia (Bezanson, Edelman, Karpinski, and Shah 2017), system tools, and dependencies.
- **rixpress** (and its Python counterpart **ryxpress**) distinguishes itself from other polyglot pipeline tools in two ways: (1) tight integration with **rix** means that the software management tool and the workflow orchestration tool are part of a single coherent framework; and (2) seamless data transfer between pipeline steps written in R, Python, or Julia is handled automatically, without the user needing to manage intermediate file formats.
- The **rstats-on-nix** fork of **nixpkgs** is a critical piece of infrastructure underlying both packages. It provides daily snapshots of CRAN and the Julia registry along with pre-built binary caches, so that users benefit from **Nix**'s reproducibility guarantees without the compilation overhead of building packages from source.

Together, these packages provide a unified approach to managing reproducible, polyglot environments and executing complex workflows with step-level isolation. While **Nix** remains complex for advanced use, **rix** and **rixpress** abstract this complexity, making deep reproducibility accessible to researchers.

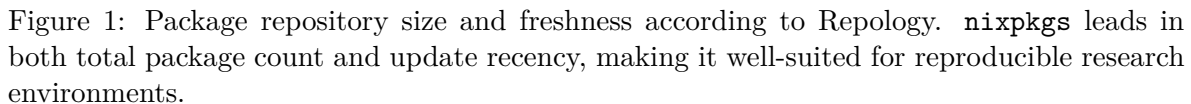
The remainder of this paper proceeds as follows. Section 2 introduces **Nix** and explains how its functional model enables reproducibility. Section 3 presents **rix** and demonstrates environment definition, including a discussion of the **rstats-on-nix** fork of the package repository. Section 4 introduces **rixpress** and demonstrates polyglot pipeline orchestration. Section 5 surveys related work and compares **rix** and **rixpress** to alternative tools in the ecosystem. Section 6 compares the declarative **Nix**-based approach with traditional imperative toolchains using a case study. Section 7 discusses limitations, and Section 8 concludes.

2. The Nix package manager

2.1. How Nix manages dependencies

Nix is a cross-platform package manager designed for reproducible software installation and deployment. Unlike traditional package managers, **Nix** emphasizes immutable infrastructure and a functional programming paradigm to ensure computational environment consistency. The primary **Nix** package collection, **nixpkgs**, provides access to over 120,000 packages, including nearly all of CRAN and Bioconductor.¹ According to Repology, a service tracking package repositories across distributions, **nixpkgs** is both the largest and among the most up-to-date repositories available (Figure Figure 1).

¹Only about 80 packages currently fail to build on the NixOS build farm, plus roughly 30 packages marked as broken by the maintainers of the R ecosystem. These could, in principle, be fixed, but doing so would require significant effort. Marking them as broken signals to users that their build failures are known and intentional. It should also be noted that some packages may build successfully but fail at runtime when attempting to install additional dependencies; one such example is **torch**.



A key advantage of **Nix** over traditional installation methods lies in its comprehensive dependency management. Consider the **sf** package for spatial data analysis in R. It requires several complex system libraries: **GDAL**, **GEOS**, and **PROJ**. Manually installing and configuring these libraries is challenging and platform-specific. With **Nix**, users simply declare **sf** as a project requirement, and **Nix** automatically installs and configures all necessary system libraries as transitive dependencies.

Comprehensive dependency tracking, however, is not unique to **Nix**. The **r2u** project by [Edelbuettel \(2025\)](#) achieves something similar for Ubuntu: once the r2u repository is configured, every **R** package is installed as a binary with all its dependencies automatically resolved. In principle, any package manager could implement such tracking.

What distinguishes **Nix** (and its sibling **Guix**, the only other functional package manager) is its *functional package* management model. This model makes full dependency specification intrinsic to the system, mandatory by design, and replaces the imperative, step-by-step nature of traditional package managers (*install this, then that*) with a declarative one: users describe the desired end state (*an environment with these packages*), and **Nix** determines how to realize it.

A further advantage is that packages built by **Nix** act as universal binaries across any Linux system (on the same architecture) where **Nix** is installed. This “build once, run anywhere” property has long been solved by **Nix**, and is now being revisited by Posit through its initiative to provide portable CRAN binaries (Posit, PBC 2025).

2.2. Derivations and reproducible builds

This universality stems from **Nix**’s use of derivations, declarative blueprints that specify all inputs (source code, build commands, dependencies) required to build a package. When software is installed, **Nix** evaluates expressions written in the Nix language to produce these derivations. Because every input is explicitly declared, builds are fully reproducible and independent of the surrounding environment.

For example, the **rJava** package definition includes the correct `JAVA_HOME` configuration pointing to its **jdk** dependency. Outside **Nix**, users must manually set this variable if automatic detection fails. Within **Nix**, maintainers encode such details once, ensuring every user benefits from a consistent, reliable installation.

At its core, **Nix** treats software builds as pure functions: identical inputs (source code, dependencies, build instructions) yield identical outputs (built packages), regardless of when or where the build occurs.

This determinism is achieved through hermetic (isolated) builds with no hidden dependencies on global system state nor undeclared side effects. The effectiveness of this model was validated at unprecedented scale by Malka, Zacchioli, and Zimmermann (2025), who rebuilt 709,816 packages from historical **nixpkgs** snapshots (2017–2023), finding rebuildability rates exceeding 99% and bitwise reproducibility rates between 69% and 91%, with an upward trend. Importantly, about 15% of unreproducible builds failed due to embedded timestamps—a solvable implementation issue rather than a fundamental model limitation.

All **Nix** package definitions are hosted in the **nixpkgs** GitHub repository. By *pinning* to a specific commit (or equivalently, a specific date), users can ensure that all installed packages derive from identical build instructions. An environment built today using a pinned revision will produce the same software versions and configurations if rebuilt years later, provided the **nixpkgs** repository remains accessible (it is archived and distributed across multiple mirrors).

Nix installs packages into a content-addressed store (`/nix/store`), where each package occupies a unique subdirectory named by a cryptographic hash of its inputs. This allows multiple R versions to coexist without conflicts.

2.3. Practical considerations

While this functional approach enhances reproducibility, it can introduce complexity for package maintainers, particularly for software downloading external assets during installation (e.g., certain Bioconductor packages). For end-users, however, this complexity is largely abstracted away by **rix** and **rixpress**, which we introduce in the following sections.

For a more in-depth technical discussion of **Nix**’s design principles, see Dolstra *et al.* (2004).

By leveraging these principles, **Nix** provides a framework that can cover the functionalities of disparate tools. For R projects, it can replace the combination of **renv** for package management and **Docker** for system-level isolation. Similarly, for Python projects, it offers a more comprehensive alternative to `requirements.txt` files and virtual environments. Furthermore, **Nix** excels at composing polyglot environments, seamlessly integrating R, Python, Julia, a $\{\text{L}^{\text{A}}\text{T}_{\text{E}}\text{X}\}$ distribution, and any of the numerous other tools available in `nixpkgs`. This enables the creation of a complete, project-specific, and deeply reproducible environment suitable for both interactive development and non-interactive, automated analysis.

However, **Nix** is not without limitations, which we discuss in detail in Section 7. Key challenges include: long-term reproducibility depends on the continued accessibility of the `nixpkgs` repository and its source packages; reproducibility on macOS is somewhat compromised by dependencies on proprietary system frameworks; Windows users must rely on WSL2; and the learning curve for the Nix language can be steep. These trade-offs are important to consider when evaluating whether **Nix** is appropriate for a given project.

To make these capabilities more approachable for R users seeking reproducible, project-specific environments without learning the full Nix language, we created the **rix** and **rixpress** packages.

3. Reproducible development environments with Nix

As mentioned, **Nix** expressions are written in the Nix programming language, which is purely functional. Here is a simple example that creates a shell environment containing version 4.3.1 of R:

```
let
  pkgs = import (fetchTarball
    "https://github.com/NixOS/nixpkgs/archive/976fa336.tar.gz"
  ) {};
  system_packages = builtins.attrValues {
    inherit (pkgs) R;
  };
in
  pkgs.mkShell {
    buildInputs = [ system_packages ];
    shellHook = "R --vanilla";
  }
```

In this expression, the `let` keyword is used to define variables. The variable `pkgs` imports the set of packages from the `nixpkgs` repository at the specified commit `976fa336`. The variable `system_packages` lists the packages to include in the environment; in this case, it is just the R programming language, along with all its dependencies and their transitive dependencies. The `mkShell` function then creates a development shell with the specified packages. The `shellHook` is set to `"R --vanilla"`, meaning that entering the shell automatically starts R in vanilla mode, ignoring any startup options.

This expression can be saved in a file called `default.nix`. The environment can then be built

on a system with **Nix** installed using the `nix-build` command.² Once the build completes, the user can enter the interactive shell with `nix-shell`, which executes the following steps:

- **Nix** reads the expression and identifies what needs to be installed;
- It checks if these packages already exist in `/nix/store`;
- If not, it downloads or builds them (with all dependencies);
- It creates a new shell environment and updates the `$PATH` to include the right binaries from `/nix/store`.
- The user is now in an interactive **Nix** shell session.

This shell contains all the packages specified in `default.nix` and can be used for development, similar to activating a virtual environment in the **Python** ecosystem.

3.1. Generating environments with **rix**

Writing **Nix** expressions can be challenging for users unfamiliar with the **Nix** language. However, the ability to define a fully reproducible development environment in a single text file and then rebuild it anywhere is highly appealing. **rix** aims to lower the barrier to adoption of **Nix** for reproducibility.

While other **Nix** front-ends such as **devenv** and **Pixi** also abstract **Nix**'s complexity, they are general-purpose tools aimed at software developers. **rix** is purpose-built for data scientists and researchers: it provides language-agnostic environment definitions that can be generated from **R** or **Python** (or the command line), and it includes an integrated workflow for bootstrapping **R** environments from scratch. This positions **rix** as a specialized interface rather than a general-purpose development tool.

Rix, an **R** package, provides the `rix()` function, which simplifies generating **Nix** expressions. It is available on CRAN and can be installed like any other **R** package. Additionally, it can bootstrap an **R** development environment on a system where **R** is not yet installed but **Nix** is available. This can be done by running (inside of a terminal):

```
$> nix-shell -I \
+   nixpkgs=https://github.com/rstats-on-nix/nixpkgs/tarball/2025-10-20 -p \
+   R rPackages.rix
```

(the `-I` flag allows one to pass a specific revision of `nixpkgs`, ensuring temporary shells are also reproducible).

This command opens a temporary **R** session with **rix** available.³ From there, users can generate new **Nix** expressions for building environments. For example, the following generates a `default.nix` file that installs **R** 4.3.1 along with the **dplyr** and **chronicler** packages:

```
R> library('rix')

R> rix(r_ver = "4.3.1",
+     r_pkgs = c("dplyr", "chronicler"),
```

²For installing **Nix**, we recommend the Determinate Systems installer: <https://determinate.systems/posts/determinate-nix-installer>

³`nix-shell -p` starts an interactive shell with the specified packages.


```
+   project_path = ".",
+   overwrite = TRUE)
```

rix can also handle more complex setups, including version pinning, TeXLive packages, GitHub packages, and IDE configuration. A complete example is provided in Appendix A.1.

Under the hood, **rix** enables version pinning (such as “**AER**@1.2-8”) by programmatically downloading the source tarball for that exact package version from the CRAN Archive (using `pkgs.fetchzip` pointing to the archive URL) and compiling it using the `buildRPackage` helper in **Nix**.

While the `@` syntax is convenient, it has critical limitations: 1. **CRAN Archive restrictions:** It cannot download the *latest* version of a package from the CRAN Archive because active package versions reside in the main CRAN `/contrib/` directory rather than the `/Archive/` directory. Thus, attempts to pin the current version of a package will return an HTTP 404 error. 2. **Dependency version mismatches:** The package is built from source using the package dependencies present in the requested `nixpkgs` snapshot, *not* the dependencies that existed when that package version was originally released on CRAN. This can lead to compilation or runtime namespace version failures (for example, attempting to compile a legacy version of **dplyr** that requires a newer version of **rlang** than is available in the selected `nixpkgs` snapshot).

Due to these compatibility challenges, manual version pinning should generally be discouraged. The recommended practice is to identify a single `nixpkgs` snapshot date where all required package versions are naturally co-compatible.

3.2. Polyglot environments and Python handling

It is also possible to include Python and Julia packages using the `py_conf` and `j1_conf` arguments respectively. Julia packages work the same way as R packages: they are pinned to their versions as of the specified date.

Python packages, however, require special handling. Unlike CRAN or the Julia package registry, PyPI (the Python Package Index) contains over 500,000 packages with highly variable quality—many lack proper build specifications, have broken dependencies, or are abandoned. Because of this heterogeneity, **Nix** maintainers cannot automatically package all of PyPI as they do for CRAN or the Julia registry. Instead, they manually curate and package individual Python packages that meet quality standards.⁴ This means some Python packages, or specific versions, may not be available through **Nix**.

For projects requiring Python packages not available in **Nix**, **rix** supports integration with **uv**, a modern Python package manager. This allows users to combine Nix’s system-level reproducibility (Python interpreter, system libraries) with **uv**’s comprehensive package availability. While this hybrid approach sacrifices some of Nix’s deterministic build guarantees for Python packages, it provides a pragmatic solution when the needed packages are unavailable in `nixpkgs`.

Users can include **uv** as a system package:

⁴On October 2025, this amounts to roughly 10000 packages, according to the search engine for **Nix** packages.


```
R> rix(date = "2025-10-20",
+   r_pkgs = c("rix", "dplyr", "chronicler"),
+   system_pkgs = c("uv"),
+   project_path = ".",
+   overwrite = TRUE)
```

This hybrid approach offers complementary strengths: **Nix** manages the Python interpreter, system libraries (e.g., **GDAL**, **HDF5**), and R packages with full determinism, while **uv** handles Python packages through its own lockfile mechanism (`uv.lock`). Within the **Nix** environment, users run standard **uv** commands (e.g., `uv pip install pandas==2.0.0`) to install Python packages. The **uv** lockfile records exact package versions and their hashes, providing reproducibility for the Python package layer.

While this approach doesn't achieve Nix's level of build-time determinism for Python packages (**uv** still downloads wheels or builds packages at install time rather than using pre-built, content-addressed artefacts) it provides a pragmatic balance: comprehensive Python package availability with reasonable reproducibility guarantees. For polyglot projects, this means R and system-level dependencies remain fully reproducible through **Nix**, while Python packages gain the reproducibility that **uv** can provide. We recommend, of course, using Python through **Nix** whenever possible and only resorting to the hybrid approach with **uv** as a last resort.⁵

3.3. Custom compiled analysis software

For workflows requiring custom compiled binaries (such as a custom C++ analysis program), users have two main integration options:

1. **Interactive compilation:** By adding standard compiler toolchains (such as `gcc`, `clang`, `gmake`, or `cmake`) to the `system_pkgs` argument of `rix()`, the resulting **Nix** shell provides a complete, hermetic environment for compiling the custom software locally. The user or a workflow script can run compilation commands (for example, `g++ -O3 -o my_analysis analysis.cpp`) inside the shell, helping ensure that the same compiler version and system libraries are used on every rebuild.
2. **Declarative packaging:** For a fully declarative approach, users can write a custom **Nix** derivation (using `stdenv.mkDerivation`) that defines the build instructions for the custom software. This custom derivation can then be imported into the `default.nix` file, allowing **Nix** to build and manage the custom software in the same content-addressed, sandboxed manner as packages from `nixpkgs`.

We recommend the second option: although it has a steeper learning curve (especially for researchers unfamiliar with the **Nix** expression language), it represents the cleanest and most reproducible approach. While **rix** does not directly automate the creation of custom derivations, the generated `default.nix` file can serve as a foundation. Generative AI tools (such as Large Language Models, or LLMs) can make writing custom derivations highly accessible. For best practices, rather than allowing an LLM to directly modify the generated `default.nix` in an ad-hoc manner, users should instruct it to write a separate **Nix** expression in its own

⁵It is also possible to package the missing Python for **Nix** and submit a pull request to `nixpkgs`, but we recognise that this option may not be beginner-friendly.

file (e.g., `custom-package.nix`), which can then be cleanly imported into or composed with the main environment.

3.4. Practical usage patterns

Anywhere **Nix** can be installed, users can benefit from its features. For instance, the repository containing the source code for this article⁶ uses GitHub Actions to compile the paper. Each time a push is made to the master branch, a runner installs **Nix**, generates the environment from the hosted `default.nix` file, and compiles the paper using **Quarto** within the reproducible environment. This helps ensure that *exactly* the same environment is used on the authors' machines and on the CI/CD platform without any additional, platform-specific, configuration.

Instead of first entering a **Nix** shell, it is also possible to run a program directly from the environment:

```
cd /path/to/project/ && nix-shell default.nix --run "Rscript analysis.R"
```

This command runs **Rscript** and executes the `analysis.R` script, which in this example should be located in the same directory as `default.nix`.

3.5. The `rstats-on-nix` fork of `nixpkgs`

When generating expressions with **rix**, the fork `rstats-on-nix/nixpkgs` is used instead of the official repository. This fork addresses a key limitation: while CRAN and Bioconductor update daily, the R packages in upstream `nixpkgs` are only updated with new R releases, as **Nix**'s governance prioritises system-level stability over rapid language ecosystem updates.

The `rstats-on-nix` fork provides daily snapshots of CRAN and the Julia registry. Each day, updated packages are committed to dated branches via GitHub Actions. Users select a specific date with `rix(date = "2024-12-14", ...)`, and can view available dates with `rix::available_dates()`. Each Monday, automated tests verify that popular packages build correctly on Linux and macOS before a date is added to this list. This infrastructure also allows us to backport fixes—for example, when RStudio was temporarily broken due to a **boost** dependency issue, we backported the fix before it was merged upstream.

As of early 2026, the fork tracks over 24,000 R packages from CRAN and Bioconductor, along with a growing set of Julia packages. Not every CRAN package is guaranteed to build successfully in **Nix** (some require system libraries that may be difficult to package), but the vast majority of commonly used packages—including the tidyverse, tidymodels, and Bioconductor workflows—are tested and available.

Because forked packages are not prebuilt by Nix's CI platform (Hydra), we provide a binary cache sponsored by Cachix to avoid lengthy source builds. Instructions for using this cache are in **rix**'s documentation.

Regarding sustainability: users are never locked into the fork—they can point directly to any upstream `nixpkgs` commit. The fork introduces no proprietary extensions; it simply provides

⁶https://github.com/b-rodrigues/rix_paper

tested, dated snapshots. Should maintenance cease, users would transition seamlessly to upstream commits, with the only trade-off being less frequent R package updates. The fork's maintenance burden is currently absorbed by the **rix** development team, with the long-term goal of upstreaming the necessary infrastructure to **nixpkgs** proper.

4. Orchestrating workflows with **rixpress**

Having established how **rix** creates reproducible *environments*, we now turn to the second challenge: reproducible *execution*. Defining a reproducible environment with **rix** addresses the first major challenge of reproducibility: environment capture. The second challenge, reliably and efficiently executing the analysis workflow within that environment, is addressed by **rixpress** (or **ryxpress** for Python users).

As mentioned in the introduction, a build automation tool like **targets** is invaluable for managing complex analyses. It tracks dependencies between code and data, caches results, and only recomputes steps that have changed. One can run a **targets** pipeline inside a **Nix** environment to make it reproducible. However, this approach has limitations: the entire pipeline must run in a single environment, and orchestrating steps across different languages (e.g., R and Python) requires manual handling via packages like **reticulate**.

rixpress overcomes these limitations by using **Nix** not just as a package manager, but as the build automation engine itself. In a **rixpress** pipeline, each step is defined as a **Nix** derivation, providing two key benefits:

1. True Polyglot Pipelines: Each step can have its own **Nix** environment. A Python step can run in a pure Python environment, an R step in an R environment, and a **Quarto** rendering step in yet another, all within the same pipeline.
2. Deep Reproducibility: Each step is a hermetically sealed **Nix** derivation whose output is cached in the **Nix** store based on the hash of all its inputs. All artefacts depend directly on their computational environment, ensuring that any change in dependencies triggers a rebuild.

4.1. Defining a polyglot pipeline

As an example, we demonstrate a polyglot pipeline that simulates a Real Business Cycle (RBC) model in Julia, trains an **XGBoost** forecasting model in Python on the simulated data, visualises results in R, and compiles a **Quarto** report. The code for this example can be found in the [following repository](https://github.com/b-rodrigues/rix_paper/tree/master/case-study)⁷, with additional details provided in the Appendix of this manuscript. Although this example is purely synthetic and not of scientific significance, it effectively demonstrates the capabilities of **rixpress**. Note that the scientific code for this synthetic example was generated by a large language model to demonstrate the framework's capabilities.

For new projects, **rixpress** provides `rxp_init()`, which generates the necessary boilerplate structure including a pipeline definition file and template helper scripts. Users then define their pipeline in the script `gen-pipeline.R` using functions inspired by **targets**, and the

⁷https://github.com/b-rodrigues/rix_paper/tree/master/case-study

computational environment the pipeline will be executed in the `gen-env.R` script. If functions are needed for pipeline steps, these are to be specified, in language-specific helper scripts: `functions.jl` for Julia, `functions.py` for Python, and `functions.R` for R. This separation keeps the pipeline definition clean and readable while encapsulating the scientific logic in appropriate language-native files. Importantly, these helper scripts are standard language files—they contain no **rixpress**-specific code and can be reused in other projects or executed independently outside of **Nix**. This design allows users to preserve standard development practices while still benefiting from declarative orchestration.

The following example demonstrates how **rixpress** orchestrates a granular, multi-step data science workflow that crosses language boundaries. (Complete setup instructions and a detailed walkthrough are provided in the Appendix.)

```
R> library('rixpress')

R> pipeline_steps <- list(
+ # STEP 0: Define RBC Model Parameters in Julia
+ rxp_jl(alpha, 0.3),
+ rxp_jl(beta, 1 / 1.01),
+ # ... (other parameters omitted for brevity) ...
+
+ # STEP 1: Julia - Simulate the RBC model
+ rxp_jl(
+   name = simulated_rbc_data,
+   expr = "simulate_rbc_model(alpha, beta, delta, rho, sigma, sigma_z)",
+   user_functions = "functions/functions.jl",
+   encoder = "arrow_write"
+ ),
+
+ # STEP 2.1: Python - Prepare features
+ rxp_py(
+   name = processed_data,
+   expr = "prepare_features(simulated_rbc_data)",
+   user_functions = "functions/functions.py",
+   decoder = "pyarrow.feather.read_feather"
+ ),
+
+ # STEP 2.2: Python - Split data (X_train, y_train, etc.)
+ rxp_py(name = X_train, expr = "get_X_train(processed_data)", ...),
+ # ... (other data splits omitted for brevity) ...
+
+ # STEP 2.3: Python - Train the XGBoost model
+ rxp_py(
+   name = trained_model,
+   expr = "train_model(X_train, y_train)",
+   user_functions = "functions/functions.py"
+ ),
+
+ )
```

```

+ # STEP 2.4: Python - Make predictions and format results
+ # ... (prediction and formatting steps omitted for brevity) ...
+ rxp_py(
+   name = final_predictions_df,
+   expr = "format_results(y_test, model_predictions)",
+   user_functions = "functions/functions.py",
+   encoder = "save_arrow"
+ ),
+
+ # STEP 3: R - Visualise the predictions
+ rxp_r(
+   name = output_plot,
+   expr = plot_predictions(final_predictions_df),
+   user_functions = "functions/functions.R",
+   decoder = arrow::read_feather
+ ),
+
+ # STEP 4: [Quarto]{.pkg} - Compile the final report
+ rxp_qmd(
+   name = final_report,
+   qmd_file = "readme.qmd"
+ )
+)

```

```

# Generate the 'pipeline.nix' file from the R list
R> rxp_populate(pipeline_steps, build = TRUE)

```

Computation steps, referred to as *derivations*, are defined using the functions `rxp_r()`, `rxp_jl()`, and `rxp_py()`, corresponding to R, Julia, and Python environments, respectively. Variants of these functions suffixed with `_file()` designate *import derivations*, which introduce external data into the pipeline. It is important to note that any data imported in this way is stored within the **Nix** store, ensuring reproducibility but also persisting the data in the build environment. The function `rxp_qmd()` is used to compile a **Quarto** document (there is also `rxp_rmd()` which serves the same purpose for **R Markdown** documents).

The `rxp_populate()` function translates this R list into a `pipeline.nix` file, which declaratively defines the entire workflow. Data flows from derivation to derivation by being serialised into the efficient and language-agnostic **Arrow** format using the pair of `encoder/decoder` functions. Other universal formats, such as `csv` or `json` could have been used. It should also be noted that if interoperability libraries such as **reticulate** or **rds2py** are available, it becomes possible to seamlessly transfer supported objects to and from R and Python (for now, such a feature is not yet integrated for conversion of arbitrary Julia objects).

As a sidenote: Python users who wish to use **ryxpress** define pipelines using the same R-based DSL shown above. This design choice keeps the pipeline definition language consistent across both R and Python ecosystems. **ryxpress** calls R and **rixpress** under the hood to generate and build the `pipeline.nix` file. To use **ryxpress**, one should simply add this package to the `py_conf` argument of `rix()`.

4.2. Executing and inspecting results

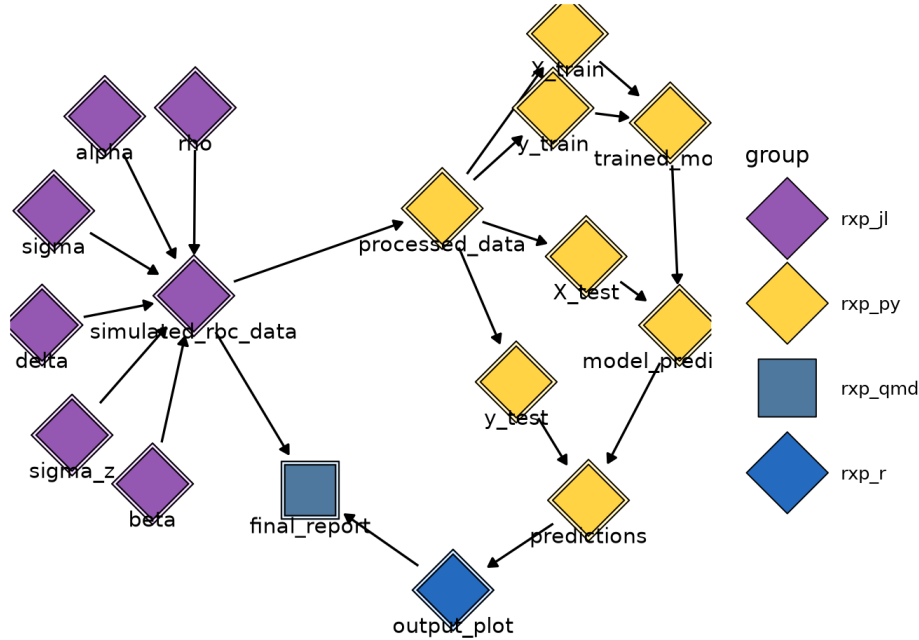


Figure 2: Graphical representation of the polyglot pipeline. Purple nodes are **Julia**, yellow nodes are **Python**, light blue nodes are **R**, and dark blue nodes are **Quarto** derivations.

The package can then generate a visual representation of the pipeline’s directed acyclic graph by running `rxp_dag()`, as can be seen in Figure 2.

To execute the pipeline, one can either set `build = TRUE` in `rxp_populate()` or call `rxp_make()` separately. **Nix** executes each step in order, building dependencies as needed. Outputs are cached, so subsequent runs only recompute steps with changed inputs or code. This provides the efficiency of **targets** with polyglot support and bit-for-bit reproducibility. Artefacts can be inspected interactively in R using `rxp_read("artefact_name")` or `rxp_load("artefact_name")`.

rixpress also includes several additional features not covered here for brevity. As mentioned previously, it is also possible to configure popular IDEs to work interactively and seamlessly with both **rix** and **rixpress**, enabling a smooth, reproducible workflow from within the development environment. Detailed setup instructions are provided in the vignettes of both packages.

In summary, **rixpress** extends the guarantees of **rix** from static environments to dynamic workflows, enabling end-to-end, polyglot reproducibility.

While GNU **Make** remains a classic tool for scientific workflows, modern pipelines are often orchestrated using dedicated engines like **Snakemake** (Mölder, Jablonski, Letcher *et al.* 2021) and **Nextflow** (Di Tommaso, Chatzou, Floden, Barja, Palumbo, and Notredame 2017). These engines support polyglot steps and per-step environments via **conda** or containerization, and have mature ecosystems for HPC schedulers (SLURM, SGE, LSF) and cloud environments.

The table below summarises the key distinction between these tools and **rixpress**: in all established engines, the execution environment is managed externally to the workflow itself, whereas in **rixpress** each pipeline node carries its own hermetically sealed environment as part of its definition.

Tool	Environment management	Workflow orchestration
Make	External	Yes
targets	Single shared environment	Yes
Snakemake	External (conda/container)	Yes
Nextflow	External (conda/container)	Yes
rixpress	Per-node, part of definition	Yes

On the one hand, **rixpress** provides a more unified experience: both the computational environments and the workflow steps are defined programmatically through a single R interface, whereas combining **Snakemake** or **Nextflow** with **conda** or **Docker** requires writing pipeline steps in a separate DSL and maintaining distinct dependency files such as Conda YAMLS or Dockerfiles. **rixpress** also handles data serialization automatically, passing objects between R, Python, and Julia steps without the user needing to manually write and read intermediate files.

On the other hand, **rixpress** has real limitations that users should be aware of. Its language support is currently limited to R, Python, and Julia: executing arbitrary command-line tools requires wrapping them in R or Python code, or defining a dedicated shell step, whereas **Snakemake** and **Nextflow** treat shell commands as first-class steps. Because **rixpress** executes pipeline steps as **Nix** derivations, intermediate inputs and outputs are stored in `/nix/store/`, which introduces storage and performance overhead for workflows with large intermediate artifacts—a consideration particularly relevant in domains such as genomics or imaging. Finally, **Snakemake** and **Nextflow** have native executors for HPC schedulers and cloud environments, whereas **rixpress** currently only supports local parallelization.

5. Related work and ecosystem comparison

To contextualise our contributions, we survey existing tools and compare their design targets and capabilities with **rix** and **rixpress**.

5.1. Package and environment management

A range of tools now exist to help researchers approach the gold standard of full reproducibility. In R, the `sessionInfo()` function reports loaded packages and versions, but reproducing the environment still requires manually installing the correct package versions—a process that becomes difficult when packages depend on system-level libraries such as **GDAL**, **GEOS**, and **PROJ** for the **sf** package.

More robust approaches define per-project environments. In Python, tools like **uv** (Astral 2024) manage interpreters and packages via lock files. In R, **renv** (Ushey and Wickham 2025) captures the project’s software state in a lock file (`renv.lock`). These tools are highly effective

within their intended scope—but neither manages the language runtime itself (e.g., the version of R) nor system-level dependencies. Other tools such as **groundhog** (Simonsohn and Gruson 2023), **rang** (Chan and Schoch 2023), and the Posit Package Manager provide date-based CRAN snapshots, but similarly focus on R packages alone. For researchers who work across multiple languages and need full-stack reproducibility, these tools must be composed with other solutions, which introduces coordination overhead.

Beyond containerization, other tools address system-level dependencies. **conda** (Anaconda Inc. 2016) is a cross-platform package and environment manager that has been widely adopted since its release in 2012, particularly in the Python and data science communities. A related tool in this space is **Pixi** (<https://prefix.dev/pixi>), which builds on the **conda** ecosystem and therefore inherits many of its advantages and limitations regarding deterministic rebuilding.

The key differentiator between **Nix** and **conda** is not mutability per se, but the nature of their reproducibility guarantees. Most **conda** users consume pre-built binary packages rather than building from source, which works well in practice but means that reproducibility depends on the continued availability of packages in the channel and on the binary being built in a consistent environment. **Nix**, by contrast, stores packages in immutable, content-addressed paths under `/nix/store/` whose names are hashes of all build inputs—source code, compiler version, build flags, and dependencies—making package identity and provenance explicit. More critically, because **conda** channels are mutable repositories, rebuilding an old environment may produce different results if packages have been updated or removed; **Nix** pins the entire repository to a specific **Git** commit, ensuring that the complete build tree—including compilers—is reconstructed exactly as it was. As discussed in Section 2 and empirically validated by Malka *et al.* (2025), this functional model achieves high rebuildability and bit-for-bit reproducibility rates even across multi-year timeframes.

Spack (Gamblin, LeGendre, Collette, Lee, Moody, de Supinski, and Futral 2015) occupies a different niche: it is aimed primarily at HPC environments, where its goal is configurability rather than reproducibility. **Spack** excels at building software with fine-grained control over compiler flags, MPI implementations, and architecture-specific optimizations. **Nix** and **Spack** therefore have largely different optimization targets—**Spack** prioritizes flexibility and architecture-specific performance, whereas **Nix** prioritizes deterministic builds and immutable dependency graphs. Researchers working on HPC systems may find **Spack** better suited to their infrastructure requirements, while **Nix** is better suited to workflows where long-term reproducibility is the primary concern. In HPC settings, **Apptainer** (formerly Singularity) addresses deployment in much the same way as **Docker**; the same distinction applies: containers package an environment, whereas **Nix** reproducibly constructs it. Beyond language-specific managers, cross-platform environment managers like **conda** (Anaconda Inc. 2016) and its modern wrapper **Pixi** (<https://prefix.dev/pixi>) handle system-level dependencies. In high-performance computing (HPC) settings, **Spack** (Gamblin *et al.* 2015) is widely used to manage dependencies with compiler-level control. Like **Nix**, these tools can manage system environment variables; for example, both **Nix** and **conda-forge** automatically configure environment variables like `JAVA_HOME` when installing packages such as **rJava** to prevent manual setup errors.

However, the reproducibility model differs fundamentally: 1. **Repository mutability**: While **conda** channels are mutable and can experience package updates or removals, **Nix** pins the entire repository to an immutable **Git** commit, ensuring the complete build tree

(including compilers) is reconstructed identically. 2. **Build hermeticity**: **conda** and **Spack** default to downloading pre-built binaries or compiling them in the host environment, whereas **Nix** enforces hermetic builds inside an isolated sandbox, ensuring builds are fully independent of host system configurations.

5.2. Containerization and execution environments

The most comprehensive approach to managing a complete computational environment to date is containerization, for which **Docker** has become the de facto standard.⁸ Docker’s principal strength lies in its capacity to package an entire data product—including the operating system, all system-level libraries, programming language runtimes, and packages—into a single, self-contained artifact known as an image. Once this image is built, the analysis can be executed within a container, which is a running instance of that image. This methodology effectively addresses the problem of missing system-level dependencies; because **Docker** images are typically minimal Linux systems, they can bundle the specific versions of libraries like **GDAL** or **GEOS** required by packages such as **sf**. These images can then be shared via public registries, enabling a high degree of reproducibility across different host systems. The Rocker project, for instance, provides invaluable pre-built **Docker** images tailored for the R community, offering a convenient foundation for reproducible research environments (Boettiger and Eddelbuettel 2017).

However, while **Docker** excels at providing this form of spatial reproducibility, its application in interactive research workflows reveals several significant limitations. First, the container model is often poorly aligned with the iterative, interactive nature of data analysis. Running graphical applications like RStudio Desktop requires complex and platform-specific configuration, while ephemeral container filesystems mean that changes made during a session are lost by default. Consequently, many research workflows develop interactively on a host system and containerize later, reproducing the deployment environment effectively but without a continuous reproducibility guarantee throughout the research process. Packages like **dockerfiler** (Fay, Guyader, Parry, and Rochette 2024) are specifically made to support this type of workflow.

Second, the effective use of **Docker** introduces a considerable learning curve. Authoring a reliable and truly reproducible **Dockerfile** requires a degree of familiarity with Linux system administration. Best practices, such as referencing base images by their immutable content digests rather than mutable tags, are necessary for long-term stability but are not widely adopted. Furthermore, while **Docker** can house polyglot environments, it offers no native mechanism for orchestrating a pipeline where distinct steps might require different, and potentially conflicting, sets of dependencies. The entire analysis is typically confined to a single, monolithic environment.

The most critical limitation, however, lies in its guarantees of temporal reproducibility. As noted by Dellaiera (2024), reproducibility is best viewed as a spectrum. **Docker** provides

⁸Container technology is standardised by the Open Container Initiative (OCI, <https://opencontainers.org>), which defines common specifications for container images and runtimes. Several OCI-compliant alternatives to **Docker** exist, notably **Podman** (<https://podman.io>), a daemonless engine that is increasingly adopted in enterprise and HPC settings, and **Apptainer** (<https://apptainer.org>), formerly known as Singularity, which is widely used in HPC environments where Docker’s root-privilege requirements are prohibitive.

strong run-time reproducibility (consistency across space), but offers weaker build-time reproducibility (consistency over time). The imperative commands within a **Dockerfile**, such as `apt-get update`, are non-deterministic by nature and can introduce temporal drift, where rebuilding the same file at different times yields different binary artifacts. As empirically demonstrated by Malka *et al.* (2025), achieving deterministic builds requires a system designed for this purpose, a role for which functional package managers—which ensure that identical inputs always produce identical outputs—like **Nix** are architecturally better suited.⁹

This is not to say that **Docker** has no place in a reproducible research workflow. Its utility for production deployment, continuous integration, and cross-platform distribution is well-established. Indeed, the two approaches are not mutually exclusive. As shall be explained in Section 2, it is possible to use **Nix** within **Docker** to benefit from the advantages of both tools.

By combining the strengths of **Nix** and container technologies, it is possible to enhance both build-time and runtime isolation. **Nix** can build container images directly, bypassing the imperative **Dockerfile** approach and producing images with stronger reproducibility guarantees. Alternatively, running a **Nix** environment inside a container can be a simpler way to integrate **Nix** with existing CI tooling.

5.3. Workflow orchestration engines

Making sure the development environment is reproducible is but one piece of the puzzle, though. Even with a perfectly reproducible environment, researchers face another challenge: reliably executing the analysis workflow. Which scripts run first? What are the dependencies between steps? Manual execution is error-prone and poorly documented. Build automation tools like **Make** address this by defining analyses as ordered, reproducible steps.

Within R, the **targets** package (Landau 2021) provides a modern, declarative approach to workflow management. It tracks dependencies, caches results, and only recomputes affected steps, improving efficiency for complex analyses. However, **targets** assumes a single software environment for the entire pipeline. For truly polyglot pipelines—where steps may require incompatible dependencies or different language versions—this becomes problematic. Running **targets** inside a **Docker** container helps ensure reproducibility but forces all steps to share one environment.

An instructive example of combining functional package management with workflow orchestration is **PiGx** (Pipelines in Genomics), a collection of genomics pipelines that uses **Guix** (a functional package manager closely related to **Nix**) for environment management and **Snake-make** for workflow execution (Wurmus, Uyar, Ober-Blöbaum, Christoph, Godschan, Göke, and Akalin 2018). While this hybrid approach helps ensure that all tools—including **Snake-make** itself—are reproducibly installed via **Guix**, the workflow execution occurs *outside* the package manager’s build system. This means the analysis code runs in the **Guix**-provided environment but does not benefit from the additional reproducibility guarantees that come from executing within the package manager’s sandboxed, hermetic (i.e., fully isolated and

⁹This contrast between the procedural, imperative commands of a **Dockerfile** and the declarative, functional model of **Nix** highlights a fundamental dichotomy in approaches to reproducibility, a concept we will examine in detail as we introduce the **Nix** package manager in Section 2.

self-contained) build process. **BioNix** (Bedő, Di Stefano, and Papenfuss 2020) takes this further by modelling bioinformatics workflow stages as pure functions directly within **Nix**, gaining build-time reproducibility guarantees. As we will show, **rixpress** follows a similar approach: each pipeline step is built within this same hermetic environment, meaning the entire analysis—not just the environment—is subject to the same deterministic build guarantees. Unlike **BioNix**, which requires users to write Nix expressions, **rixpress** provides an accessible R (or Python) interface for defining pipelines.

Guix deserves a direct comparison. It provides many of the same theoretical guarantees as **Nix**: functional package management, content-addressed storage, and hermetic builds. Our contribution is therefore not a new package manager, but making the **Nix** ecosystem accessible to R users through **rix**, and extending it with an R-native interface for polyglot workflow orchestration through **rixpress**. The choice of **Nix** over **Guix** as the underlying system reflects the maturity and size of the **nixpkgs** package collection—one of the largest in existence—and the active community maintaining R and Python packages within it.

The current state-of-the-art in reproducible research exemplifies the complexity of navigating this fragmented landscape. The work of McDermott (2021) provides an excellent case study in best practices, with an accompanying repository that is fully reproducible.¹⁰ Achieving this level of reproducibility required the integration of multiple complementary tools: **renv** for managing R packages, **Docker** for containerizing the system environment, and **Make** for orchestrating the analysis. Each tool is effective at its specific task, but their coordination demands expertise across multiple domains and careful maintenance of the glue code that connects them.

Furthermore, the complexity increases for polyglot projects, which must manage multiple ecosystems simultaneously. While GNU **Make** remains a classic tool for scientific workflows, modern polyglot workflow engines like **Snakemake** (Mölder *et al.* 2021) and **Nextflow** (Di Tommaso *et al.* 2017) support separate computational environments per step using **conda** or containerization.

rixpress differs from these workflow engines in several ways: 1. **Unified declarative API**: By using **rix** and **rixpress** together, environment definition and workflow orchestration are handled through a single, unified R interface, avoiding the need to write separate Docker files, Conda environment files, and workflow DSLs (such as Snakemake’s Python dialect or Nextflow’s Groovy-based DSL). 2. **Implicit dependency inference**: **rixpress** automatically infers inter-step dependencies from the functional relationships in the R code, rather than requiring users to manually define file inputs and outputs for every target. 3. **Seamless cross-language data transfer**: Data is passed between R, Python, and Julia steps automatically using the **Arrow** format, without requiring manual serialization wrapper code.

However, **rixpress** has distinct limitations compared to mature engines: - **Ecosystem limits**: It is designed specifically for R, Python, and Julia. Workflow steps that require command-line utilities or other languages cannot be easily integrated, whereas Snakemake and Nextflow treat arbitrary shell commands as first-class steps. - **Storage and I/O overhead**: Because steps run as pure Nix derivations, **rixpress** copies all input and output data into the `/nix/store/` for caching. This introduces significant storage and performance overhead for workflows processing large datasets (e.g., gigabytes or terabytes). - **HPC and cloud support**: **rixpress**

¹⁰See <https://github.com/grantmcdermott/skeptic-priors>

only supports local parallelization, whereas Snakemake and Nextflow have native executors for cluster schedulers (SLURM, SGE, LSF) and cloud environments.

6. Comparison of pipeline paradigms: Imperative vs. declarative

To illustrate the practical benefits of our framework, we implemented the RBC pipeline example from Section 4 using two distinct paradigms. The first is a traditional, imperative, stack combining **Docker** for environment isolation, **Make** for workflow orchestration, and language-specific package managers.¹¹ The second is our proposed declarative framework using **rix** and **rixpress**. While both approaches successfully execute the same polyglot analysis to produce identical outputs, their comparison illuminates fundamental differences in conceptual models, system complexity, and the nature of the reproducibility guarantees they provide.

6.1. The imperative approach (Docker + Make)

The traditional approach represents a composition of specialized tools, each solving part of the reproducibility puzzle. This paradigm is purely imperative; the researcher must explicitly script the procedural steps for both environment construction and workflow execution.

The codification of the environment is a procedural build process specified in a **Dockerfile**. Successfully authoring such a file necessitates that the researcher act as a system administrator, manually scripting a sequence of shell commands to install each component. This includes managing **apt** repositories for R, invoking external installers like **uv** and **rig** for Python and R runtimes, and using **curl** and **tar** for Julia. Subsequently, separate commands must be run to install packages within each language’s ecosystem. The procedural verbosity and intertwined concerns of this approach are reflected in a **Dockerfile** spanning 108 lines to define the complete environment.

Here is a condensed sketch of this **Dockerfile**:

```
# Install R
RUN curl -L https://rig.r-pkg.org/... | sh && rig add 4.5.1
# Install Python
RUN curl -LsSf https://astral.sh/uv/install.sh | sh && uv python install 3.13
# Install Julia
RUN curl -fsSL https://julialang-s3.julialang.org/... -o julia.tar.gz && \
    tar -xzf julia.tar.gz -C /opt/
# Install packages for each language separately
RUN Rscript -e 'install.packages(...)'
RUN uv pip install --no-cache pandas==2.3.3 scikit-learn==1.7.2
RUN julia -e 'using Pkg; Pkg.add(name="Arrow", version="2.8.0")'
```

This procedural philosophy extends from environment definition to workflow orchestration. A

¹¹This choice of tools is illustrative: we could have used, for instance, **Snakemake** (Mölder *et al.* 2021) for workflow orchestration or **conda** (Anaconda Inc. 2016) instead of **uv**, but the broader point remains—these are all imperative approaches.

Makefile defines the pipeline as a directed acyclic graph of file dependencies. While robust, this model requires the researcher to manually encode the dependency graph, tightly coupling the analytical logic to specific file paths. Furthermore, since **Make** is agnostic to the content of these files, it provides no mechanism to enforce type or schema consistency between steps.

A direct consequence of this file-based orchestration model is the proliferation of wrapper scripts. Each step in the **Makefile** invokes a script containing a significant amount of procedural boilerplate unrelated to the scientific logic. This includes parsing command-line arguments, handling data serialization and deserialization, and managing file paths. The core analytical logic is thus obscured by necessary but extraneous scaffolding.

The cumulative effect is a system of considerable structural complexity. The traditional implementation requires nine separate files to fully specify the environment, workflow, and analysis. This includes the **Dockerfile**, **Makefile**, three wrapper scripts for orchestration, and three **functions** files containing the core logic, in addition to the final report. Each tool in this stack is mature and well-documented, but a user must comprehend the conventions and interactions of the entire toolchain.

6.2. The declarative approach (Nix + rix + rixpress)

The Nix-based framework inverts this imperative model. Instead of scripting the procedural construction of the environment and workflow, the researcher declares the desired end-state.

Environment specification is reduced to a single declarative statement in the **gen-env.R** script. The researcher specifies *what* the environment should contain, leaving the procedural implementation of *how* to build it entirely to the **Nix** toolchain. The **gen-env.R** script replaces the 108-line **Dockerfile**, abstracting away all system administration details. **rix** translates this high-level specification into a formal **Nix** expression that deterministically resolves all transitive dependencies for the complete polyglot environment.

```
R> library(rix)
R> rix(
+   date = "2025-10-14",
+   r_pkgs = c("ggplot2", "dplyr", "arrow"),
+   jl_conf = list(jl_version = "lts", ...),
+   py_conf = list(py_version = "3.13", ...),
+   ...
+ )
```

Workflow orchestration undergoes a similar conceptual shift from procedural to declarative. The **gen-pipeline.R** script defines the pipeline not as a set of file rules, but as a graph of data objects. Dependencies are not manually specified but are automatically inferred by **rixpress** from the functional relationships in the code. This object-centric model renders the procedural wrapper scripts unnecessary. The scientific logic is expressed as a set of pure functions that accept data structures as inputs and return them as outputs, free of any boilerplate for file I/O or command-line parsing.

```
# functions/functions.py
>>> def train_and_predict_output(simulated_df: pd.DataFrame) -> pd.DataFrame:
```



```
+    # ... scientific logic ...
+    return format_results(predictions)
```

This declarative approach significantly reduces system complexity. The entire implementation requires only six files: two for the high-level environment and pipeline declarations, three containing the pure scientific functions, and the final report. The reduction is not merely quantitative; the remaining files are conceptually simpler, containing only analytical logic and specifications, with all procedural scaffolding abstracted away by the framework.

6.3. Spatial vs. temporal reproducibility

The critical distinction between the two paradigms lies in the nature of their reproducibility guarantees. The traditional stack, anchored by Docker, provides strong **spatial reproducibility**—the ability to execute the analysis identically across different machines. However, it is vulnerable to **temporal drift**, a concept that distinguishes reproducibility across space from consistency over time (Malka, Zacchiroli, and Zimmermann 2024). This temporal vulnerability arises from several sources, including the resolution of mutable base image tags (e.g., `ubuntu:24.04`) and the potential for language-level package managers to resolve dependencies differently over time.

The Nix-based approach is designed to address this problem of temporal drift. By pinning the entire software ecosystem to a single, immutable revision of the `nixpkgs` repository, the framework provides strong guarantees of **temporal reproducibility**. This claim is supported by large-scale empirical evidence; a study by Malka *et al.* (2025), which rebuilt over 700,000 historical packages, confirmed that Nix’s functional model achieves a bit-for-bit reproducibility rate exceeding 90% and a rebuildability rate over 99% across a multi-year time-frame.

6.4. Summary of differences

Dimension	Traditional Stack	Nix-Based Framework with rix and rixpress
Paradigm	Imperative composition of specialized tools	Declarative specification in unified framework
Environment setup	Step-by-step shell commands	High-level specification
Workflow definition	File-based rules in Make syntax	Object-based graph in R syntax
Scientific code	Wrapped in CLI parsing and file I/O boilerplate	Pure functions with no plumbing
File count	9 files (4 for orchestration and plumbing)	6 files (2 for orchestration)
Expertise required	Linux system administration, Make syntax	R programming
Spatial reproducibility	Strong	Strong

Dimension	Traditional Stack	Nix-Based Framework with rix and rixpress
Temporal reproducibility	Moderate (vulnerable to drift)	Strong

This declarative rigor, however, is not without pragmatic costs. Our validation on GitHub Actions shows that the imperative Docker-based runner completes the entire pipeline in approximately three minutes, whereas the declarative Nix-based runner takes around five minutes.¹² Nix’s deterministic dependency resolution is designed for bit-for-bit reproducibility at the expense of initial computation time, whereas the imperative system prioritizes immediacy over long-term stability.

7. Limitations

While **rix** and **rixpress** address significant challenges in reproducible research, several limitations should be acknowledged.

Platform constraints. On macOS, **Nix**’s reproducibility guarantees are somewhat compromised by dependencies on proprietary system frameworks and Xcode toolchains that reside outside the **Nix** store; macOS users may need to update project pins following system upgrades. On Windows, **Nix** operates through the Windows Subsystem for Linux (WSL2), which adds configuration overhead. Long-term reproducibility also depends on the continued accessibility of the **nixpkgs** repository and the source code for all packages in the dependency tree.

Ecosystem coverage. While **nixpkgs** is extensive, not all Python packages from PyPI are available, and some R packages may fail to build. The **rstats-on-nix** fork addresses the CRAN freshness problem, but packages may experience a short lag before they are built and indexed. The hybrid approach with **uv** for missing Python packages sacrifices some of **Nix**’s deterministic build guarantees.

Operational challenges. Debugging a failed package compilation inside a sandboxed **Nix** build can be more difficult than standard R package installation, as build logs may be less familiar. Build times can be substantial for large environments when pre-built binary artifacts are not available from a cache, and the immutable **Nix** store leads to higher disk usage compared to traditional package managers. **rixpress** pipelines lack features such as automatic detection of outdated derivations, and there is currently no support for HPC scheduler integration.

Maintenance overhead. The **rstats-on-nix** fork requires ongoing maintenance to keep pace with upstream **nixpkgs** and CRAN. While the fork introduces no proprietary extensions, its long-term sustainability depends on continued community contribution.

¹²To run the benchmark yourself, fork the paper’s repository, enable GitHub Actions, and push a single change to trigger the runs.

8. Conclusion

Many tools exist to improve reproducibility, but **Nix** stands out because it deploys complete software environments closed under the “depends on” relation: it installs not only a package but all of its transitive dependencies. This makes **Nix** uniquely suited for reproducible research.

Yet solving such a complex problem makes **Nix** itself complex, as it relies on a functional paradigm still likely unfamiliar to most researchers. **rix** lowers this barrier for R users by providing a familiar interface and workflow. By declaratively specifying reproducible development shells with **Nix**, researchers can accommodate diverse use cases—from running analytical pipelines to developing interactive **shiny** applications or serving **plumber** APIs.

Furthermore, **rixpress** extends this declarative model to entire analysis pipelines. By leveraging **Nix** as a build automation system, **rixpress** enables polyglot workflows where each step runs in a hermetically sealed environment. This supports both spatial and temporal reproducibility, efficient caching, and seamless orchestration of multi-language analyses. Crucially, it provides native polyglot support without the manual orchestration required by container-based approaches, making complex workflows accessible without systems administration expertise.

Adopting **Nix**-based workflows represents a conceptual shift from procedural scripting to declarative specification. A practical adoption pathway begins with using **rix** to generate reproducible environments for new projects while maintaining existing workflows. This allows gradual learning of **Nix** concepts without disrupting active research. As confidence grows, researchers can extend to **rixpress** for smaller analyses before tackling complex pipelines. Institutions can support adoption by offering workshops, maintaining binary caches, and designating local experts. While early projects require investment, later projects benefit from reusable environment definitions and cumulative expertise. Ultimately, adoption depends on project scope, reproducibility needs, and institutional context—but the framework now exists for those who require robust, long-term reproducibility.

Acknowledgments

We thank the rOpenSci reviewers and contributors who provided valuable feedback on the development of **rix** and **rixpress**. In particular, we are grateful to David Watkins and Jacob Wujciak-Jens for their reviews of **rix**, and to William Landau and Anthony Martinez for their reviews of **rixpress**. We also acknowledge the contributions of Richard J. Acton, Jordi Rosell, Elio Campitelli, László Kupcsik, and Michael Heming for **rix**. Their expertise and feedback greatly improved the quality and usability of these packages. We would also like to thank Pol Dellaiera, Edvin Syk, Matthias Spichiger, and Jadey Ryan for providing feedback on the manuscript.

Software availability

The **rix** package is available from CRAN (<https://cran.r-project.org/package=rix>) and rOpenSci (<https://docs.ropensci.org/rix/>). The **rixpress** package is available from

CRAN (<https://cran.r-project.org/package=rixpress>) and rOpenSci (<https://github.com/ropensci/rixpress/>). The **ryxpress** package (Python port) is available from PyPI (<https://pypi.org/project/ryxpress/>). All packages are released under the GPL-3 license. The source code for this manuscript and all replication materials are available at https://github.com/b-rodrigues/rix_paper.

References

- Anaconda Inc (2016). *Anaconda Software Distribution*. URL <https://docs.anaconda.com/>.
- Astral (2024). “uv: An Extremely Fast Python Package and Project Manager.” <https://github.com/astral-sh/uv>. Accessed: 2025-10-28; written in Rust and designed as a drop-in replacement for pip/pip-tools/virtualenv.
- Bedő J, Di Stefano L, Papenfuss AT (2020). “Unifying package managers, workflow engines, and containers: Computational reproducibility with BioNix.” **9**(11), g1aa121. doi:10.1093/gigascience/g1aa121.
- Bezanson J, Edelman A, Karpinski S, Shah VB (2017). “Julia: A fresh approach to numerical computing.” *SIAM Review*, **59**(1), 65–98. doi:10.1137/141000671.
- Bhandari Neupane J, Neupane RP, Luo Y, Yoshida WY, Sun R, Williams PG (2019). “Characterization of Leptazolines A–D, Polar Oxazolines from the Cyanobacterium *Leptolyngbya* sp., Reveals a Glitch with the “Willoughby–Hoye” Scripts for Calculating NMR Chemical Shifts.” *Organic Letters*, **21**(20), 8449–8453. doi:10.1021/acs.orglett.9b03216.
- Boettiger C, Eddelbuettel D (2017). “An Introduction to Rocker: Docker Containers for R.” *The R Journal*, **9**(2), 527–536. doi:10.32614/RJ-2017-065.
- Chan Ch, Schoch D (2023). “rang: Reconstructing Reproducible R Computational Environments.” *PLOS ONE*, **18**(6), e0286761. doi:10.1371/journal.pone.0286761.
- Chen A, Wong C, Sharif B, Peruma A (2025). “Exploring Code Comprehension in Scientific Programming: Preliminary Insights from Research Scientists.” doi:10.48550/arXiv.2501.10037. 2501.10037.
- De Paoli B (2009). “Slides 1: The RBC Model, Analytical and Numerical solutions.” https://personal.lse.ac.uk/depaoli/RBC_slides1.pdf. Lecture Slides.
- Dellaiera P (2024). *Reproducibility in Software Engineering*. Master’s thesis, University of Mons, Mons, Belgium. doi:10.5281/zenodo.12666898.
- Di Tommaso P, Chatzou M, Floden EW, Barja PP, Palumbo E, Notredame C (2017). “Nextflow enables reproducible computational workflows.” *Nature Biotechnology*, **35**(4), 316–319. doi:10.1038/nbt.3820.
- Dolstra E, De Jonge M, Visser E (2004). “Nix: A Safe and Policy-Free System for Software Deployment.” In *18th Large Installation System Administration Conference*, pp. 79–92.
- Eddelbuettel D (2025). “r2u: CRAN as Ubuntu Binaries.” Accessed: 2025-10-28, URL <https://eddelbuettel.github.io/r2u/>.

- Fay C, Guyader V, Parry J, Rochette S (2024). **dockerfiler**: *Easy Dockerfile Creation from R*. R package version 0.2.5, URL <https://thinkr-open.github.io/dockerfiler/>.
- Gamblin T, LeGendre M, Collette MR, Lee GL, Moody A, de Supinski BR, Futral S (2015). “The Spack package manager: bringing order to HPC software chaos.” In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pp. 1–12. doi:10.1145/2807591.2807623.
- Landau WM (2021). “The **targets** R Package: A Dynamic Make-like Function-Oriented Pipeline Toolkit for Reproducibility and High-Performance Computing.” *Journal of Open Source Software*, **6**(57), 2959. doi:10.21105/joss.02959.
- Malka J, Zacchiroli S, Zimmermann T (2024). “Reproducibility of Build Environments through Space and Time.” In *Proceedings of the 2024 ACM/IEEE 44th International Conference on Software Engineering: New Ideas and Emerging Results*, ICSE-NIER’24, p. 97–101. ACM. doi:10.1145/3639476.3639767.
- Malka J, Zacchiroli S, Zimmermann T (2025). “Does Functional Package Management Enable Reproducible Builds at Scale? Yes.” In *22nd International Conference on Mining Software Repositories*. Ottawa, Canada. URL <https://hal.science/hal-04913007>.
- McDermott GR (2021). “Skeptic Priors and Climate Consensus.” *Climatic Change*, **166**(1-2), 7. doi:10.1007/s10584-021-03089-x.
- Mölder F, Jablonski KP, Letcher B, *et al.* (2021). “Sustainable data analysis with Snakemake.” *F1000Research*, **10**, 33. doi:10.12688/f1000research.29032.3.
- Navarro ALG, Koneva N, Sánchez-Macián A, Hernández JA (2024). “A Comprehensive Guide to Combining R and Python code for Data Science, Machine Learning and Reinforcement Learning.” doi:10.48550/arXiv.2407.14695. 2407.14695.
- Osborne I, Pivarski J, Ling J (2024). “Bridging Worlds: Achieving Language Interoperability between Julia and Python in Scientific Computing.” doi:10.48550/arXiv.2404.18170. 2404.18170.
- Peng B, Wang G, Ma J, Leong MC, Wakefield C, Melott J, Chiu Y, Du D, Weinstein JN (2018). “SoS Notebook: an interactive multi-language data analysis environment.” *Bioinformatics*, **34**(21), 3768–3770. doi:10.1093/bioinformatics/bty405.
- Peng RD (2011). “Reproducible Research in Computational Science.” *Science*, **334**(6060), 1226–1227. doi:10.1126/science.1213847.
- Posit, PBC (2025). “Introducing Portable Linux R Binary Packages.” Accessed: 2025-10-28, URL <https://posit.co/blog/introducing-portable-linux-r-binary-packages/>.
- Rodrigues B (2025). *rixpress: Build Reproducible Analytical Pipelines with Nix*. R package version 0.10.1, URL <https://github.com/ropensci/rixpress/>.
- Rodrigues B, Baumann P (2025). *rix: Reproducible Data Science Environments with Nix*. R package version 0.17.2, URL <https://docs.ropensci.org/rix/>.

- Simonsohn U, Gruson H (2023). *groundhog: Version-Control for CRAN, GitHub, and GitLab Packages*. R package version 3.1.2, <https://github.com/CredibilityLab/groundhog>, URL <https://groundhogr.com/>.
- Ushey K, Wickham H (2025). *renv: Project Environments*. R package version 1.1.5, URL <https://rstudio.github.io/renv/>.
- Wickham H (2016). *ggplot2: Elegant Graphics for Data Analysis*. Springer-Verlag New York. ISBN 978-3-319-24277-4. URL <https://ggplot2.tidyverse.org>.
- Wurmus R, Uyar B, Ober-Blöbaum B, Christoph J, Gosdschan A, Göke J, Akalin A (2018). “PiGx: Reproducible genomics analysis pipelines with GNU Guix.” *GigaScience*, **7**(12), giy123. doi:10.1093/gigascience/giy123.

9. Appendix

9.1. Reproducing this paper

The source code of this paper is hosted on GitHub and can be found at following [link¹³](#). The paper can be easily compiled by running the following command:

```
$> nix-shell --run "quarto render paper.qmd --to jss-pdf"
```

The `default.nix` file defines the exact computational environment required to compile the manuscript, promoting full reproducibility of the build process. To help ensure the reproducibility of the manuscript, it is automatically recompiled via GitHub Actions upon each commit, using the same **Nix** environment. An HTML version is additionally deployed to GitHub Pages, providing an accessible format for viewing on smaller devices at this [link¹⁴](#). The PDF version of the manuscript can be found at [link¹⁵](#).

9.2. A complete polyglot example with rixpress

This appendix provides a conceptual walk-through of the polyglot pipeline example discussed in Section 5. The pipeline simulates a RBC model in Julia, trains an **XGBoost** model in Python, visualises the results in R, and compiles a final report with **Quarto**.

The complete, runnable source code for this example is available in the paper’s GitHub repository. A shell script, `repro_script.sh`, is provided to automate the entire process from environment creation to final output, as described at the end of this section¹⁶.

Project structure and components

The project is organized into a set of scripts, each with a distinct responsibility. The core logic is separated from the environment definition and pipeline orchestration.

¹³https://github.com/b-rodrigues/rix_paper

¹⁴https://b-rodrigues.github.io/rix_paper/

¹⁵https://b-rodrigues.github.io/rix_paper/paper.pdf

¹⁶https://github.com/b-rodrigues/rix_paper/blob/master/repro-script/repro_script.sh

- **functions/**: This directory contains the helper scripts with the core analytical code for each language.
 - **functions.jl**: The Julia script for the economic simulation.
 - **functions.py**: The Python script for the machine learning workflow.
 - **functions.R**: The R script for data visualisation.
- **gen-env.R**: An R script that defines the reproducible computational environment.
- **gen-pipeline.R**: The main R script that defines and orchestrates the entire polyglot pipeline.

Step 1: The environment definition

The foundation of the project is the reproducible environment, defined in **gen-env.R**. This script uses the **rix()** function to programmatically generate a **default.nix** file. This file serves as a complete blueprint for the computational environment, specifying:

- The exact versions of R, Python, and Julia.
- A list of required packages for each language (e.g., **ggplot2** (Wickham 2016) for R, **xgboost** for Python, **DataFrames** for Julia).
- System-level dependencies like **Quarto**.

Crucially, the entire environment is pinned to a specific date (2025-10-14), ensuring that anyone who builds the environment, now or in the future, will get the exact same software versions, promoting reproducibility.

By dropping into a temporary shell using the following command:

```
$> nix-shell -I \
+ nixpkgs=https://github.com/rstats-on-nix/nixpkgs/tarball/2025-10-20 -p \
+ R rPackages.rix
```

it is possible to generate the **default.nix** by sourcing **gen-env.R**. Then, one leaves this temporary shell, and builds the environment using the command **nix-shell**, which also drop the user into the development shell.

Step 2: The core analytical logic

The scientific logic for each stage of the pipeline is encapsulated in the separate helper scripts within the **functions/** directory.

- The RBC Model Simulation (**functions.jl**): This Julia script contains a single pure function that implements the state-space solution to the RBC model, based on De Paoli (2009). It takes the model's economic parameters as inputs and returns the simulated time-series data as a **DataFrame**.
- The **XGBoost** Forecasting Model (**functions.py**): This Python script handles the complete machine learning workflow through a series of modular functions. Its responsibilities include feature engineering (creating lagged variables), splitting the data into training and testing sets, training the **XGBoost** model, and generating predictions.
- The Visualisation (**functions.R**): This R script contains a function that uses **ggplot2** to create the final visualisation. It is designed to take the data frame of actual and

predicted values produced by the Python script and generate a plot comparing the two series.

Step 3: Orchestrating the pipeline

The entire workflow is defined and orchestrated by `gen-pipeline.R`. This script acts as the master plan, using functions from the `rixpress` package to define each computational step as a *derivation*. It declaratively outlines the dependencies between steps:

1. It begins by defining the RBC model parameters in `Julia`.
2. It specifies that the RBC simulation in `functions.jl` depends on these parameters.
3. It then defines the series of `Python` steps (feature preparation, training, prediction), making each one dependent on the output of the previous one. The first `Python` step is explicitly made dependent on the output from the `Julia` simulation. `rixpress` handles the passing of data between languages, in this case using the `Arrow` file format for efficiency.
4. Finally, it defines the **R visualisation** step, which depends on the final predictions from the `Python` model, and a **Quarto rendering** step that depends on the generated plot.

When this script is run in the development shell (by executing `source("gen-pipeline.R")`), `rixpress` translates the declared pipeline into a master Nix expression that `Nix` can execute, automatically handling the caching of results and re-running only the necessary steps if a piece of code or data changes.

To build the pipeline from an interactive `Python` session, one would execute the following lines:

```
>>> from rixpress import rxp_make
>>> rxp_make()
```

Running the project

This entire example can be executed by running the `repro-script.sh` script available in the `repro-script/` directory of the paper's repository. This script automates the full process: 1. It first executes `gen-env.R` inside a temporary `Nix` shell to build the `default.nix` file. 2. It then executes `gen-pipeline.R` inside the newly defined environment. This triggers `Nix` to run the entire polyglot pipeline in the correct order.

Upon completion, the script will have generated all intermediate artefacts and the final `readme.html` report found in the `pipeline-output` folder containing the visualisation.

Affiliation:

Bruno Rodrigues
Department of Statistics
18, Montée de la Pétrusse
Luxembourg Luxembourg
E-mail: bruno@brodrigues.co
URL: <https://www.brodrigues.co>

Philipp Baumann
Data and Analytics
7, Länggassstrasse
Bern Switzerland
E-mail: baumann-philipp@protonmail.com